COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	July 21, 2026
NAME:	Benj Federic C. De Leon	**INSTRUCTOR:**	Engr. Jimlord M. Quejadoimlord

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays.

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
```

```
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
Student student1("Roman", 28);
Student student2(student1);
Student student3;
student3 = student2;
return 0;

    return 0;
}
```

OUTPUT:

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

The program reads main() from top to bottom, sequentially constructing the name and age for **student1** (Constructor Called.), then cloning those attributes into **student2** (Copy Constructor Called), and finally building **student3** using default data (Constructor Called.). After the line student3 = student2; quietly overwrites data without creating a new object, the program finishes reading main() and must clear the memory stack. it does not deconstruct them in the order they were made, but instead deconstructs **student3 first, student2 second, and student1 last**.

Table 2-1. Initial Driver Program

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
```

```
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

The program starts by executing main() sequentially. It initializes your constant size variable j to 5. Then, the program reads your studentList array declaration. It immediately moves to the first index (index 0) and runs the constructor. After getting all the initial info, it prints Constructor Called. While the program is running the constructors, it defaults the attributes inside the studentList to **"John Doe"** and **18**. Right after creating the students, it reads and sets up your namesList and ageList arrays on the stack. After executing everything inside main(), the program reaches return 0;. It must now clear all variables off the memory stack. It targets the 5 students and executes the destructors 5 consecutive times.

Table 2-2. Modified Driver Program with Student Lists

INPUT:

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
```

```
Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
Student *ptr = new Student(namesList[i], ageList[i]);
studentList[i] = *ptr;
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

OUTPUT:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
```

```
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

In **Loop A**, the program runs five times and uses the word **new** to build a temporary, custom student out in extra storage space (the heap) for each name and age in your lists, which prints Constructor Called. five extra times. It then copies that new information directly into the blank student already waiting on your main table (the stack), but because the code doesn't use the word delete, these temporary storage students are left behind as a memory leak. Right after that, **Loop B** runs five times to read your main table and successfully prints out the updated names and ages (like "Carly 15" and "Freddy 16") on your screen just before the program closes and cleans up the table.

Table 2-3. Final Driver Program

INPUT (Modified):

```
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
public:
    //constructor
Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
};
//deconstructor
~Student(){
        std::cout << "Destructor Called." << std::endl;
}

    //Copy Constructor
Student(const Student &copyStudent){
```

```
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
}

    //Display Attributes
void printDetails(){
        std::cout << this->studentName << " " << this->studentAge <<
std::endl;
}

};

int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};
for(int i = 0; i < j; i++){ //loop A
    // Directly overwrite the stack student with the custom name and age
    studentList[i] = Student(namesList[i], ageList[i]);
}
for(int i = 0; i < j; i++){ //loop B
studentList[i].printDetails();
}
return 0;
}
```

OUTPUT (Modified):

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Code Analysis:

By changing Loop A to avoid the new keyword, the code prevents objects from being abandoned in heap storage, allowing C++ to automatically create a temporary student on the stack, copy its data into the array, and delete it immediately.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List		
Apple	PHP 10	X7
Banana	PHP 10	X8
Broccoli	PHP 60	X12
Lettuce	PHP 50	X10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.
Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.
Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.
Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.
.Problem 5: Discuss the

INPUT:

```
#include <iostream>
#include <string>

// Fruit class
class Fruit {
public:
    std::string name;
    double price;       // (PHP)
    int quantity;

    // Constructor
    Fruit(std::string n = "", double p = 0.0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
        std::cout << "Fruit created: " << name << std::endl;
    }

    // Copy Constructor
    Fruit(const Fruit &other) {
        name = other.name;
```

```
        price = other.price;
        quantity = other.quantity;
        std::cout << "Fruit Copied: " << name << std::endl;
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit &other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            std::cout << "Fruit assigned: " << name << std::endl;
        }
        return *this;
    }

    // Destructor
    ~Fruit() {
        std::cout << "Fruit deleted: " << name << std::endl;
    }

    double calculateSum() {
        return price * quantity;
    }

    void display() {
        std::cout << "   [Fruit] " << name
                  << " | Price: PHP " << price
                  << " | Qty: " << quantity
                  << " | Subtotal: PHP " << calculateSum() << std::endl;
    }
};

// Vegetable class
class Vegetable {
public:
    std::string name;
    double price;
    int quantity;

    // Constructor
    Vegetable(std::string n = "", double p = 0.0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
```

```
        std::cout << "Vegetable created: " << name << std::endl;
    }

    // Copy Constructor
    Vegetable(const Vegetable &other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
        std::cout << "Vegetable copied: " << name << std::endl;
    }

    // Copy Assignment Operator
    Vegetable& operator=(const Vegetable &other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            std::cout << "Vegetable assigned: " << name << std::endl;
        }
        return *this;
    }

    // Destructor
    ~Vegetable() {
        std::cout << "Vegetable deleted: " << name << std::endl;
    }

    double calculateSum() {
        return price * quantity;
    }

    void display() {
        std::cout << "   [Vegetable] " << name
                  << " | Price: PHP " << price
                  << " | Qty: " << quantity
                  << " | Subtotal: PHP " << calculateSum() << std::endl;
    }
};

// Total sum calculation helper
double Total_sum(Fruit* fruitList[], int fruitSize, Vegetable* vegList[], int vegSize) {
    double total = 0.0;

    for (int i = 0; i < fruitSize; i++) {
        if (fruitList[i] != nullptr) {
```

```
            total += fruitList[i]->calculateSum();
        }
    }

    for (int i = 0; i < vegSize; i++) {
        if (vegList[i] != nullptr) {
            total += vegList[i]->calculateSum();
        }
    }

    return total;
}

int main() {
    // Allocation
    Fruit* fruitList[2];
    fruitList[0] = new Fruit("Apple", 10, 7);
    fruitList[1] = new Fruit("Banana", 10, 8);

    Vegetable* vegList[2];
    vegList[0] = new Vegetable("Broccoli", 60, 12);
    vegList[1] = new Vegetable("Lettuce", 50, 10);

    // Initial Display
    std::cout << "\nJenna's Grocery List:" << std::endl;
    for (int i = 0; i < 2; i++) {
        if (fruitList[i] != nullptr) fruitList[i]->display();
    }
    for (int i = 0; i < 2; i++) {
        if (vegList[i] != nullptr) vegList[i]->display();
    }

    std::cout << "\nTotal: PHP " << Total_sum(fruitList, 2, vegList, 2) << std::endl;

    // Delete Lettuce from list and safely de-allocate memory
    std::cout << "\nRemoving Lettuce:" << std::endl;
    delete vegList[1];
    vegList[1] = nullptr;

    // Display list after removal
    std::cout << "\nJenna's Grocery List (Updated):" << std::endl;
    for (int i = 0; i < 2; i++) {
        if (fruitList[i] != nullptr) fruitList[i]->display();
    }
    for (int i = 0; i < 2; i++) {
```

```
        if (vegList[i] != nullptr) vegList[i]->display();
    }

    std::cout << "\nTotal (after removing Lettuce): PHP " << Total_sum(fruitList, 2, vegList, 2) << std::endl;

    //Memory de-allocation.
    std::cout << "\nProgram terminating. Cleaning remaining memory:" << std::endl;
    for (int i = 0; i < 2; i++) {
        if (fruitList[i] != nullptr) {
            delete fruitList[i];
            fruitList[i] = nullptr;
        }
    }
    for (int i = 0; i < 2; i++) {
        if (vegList[i] != nullptr) {
            delete vegList[i];
            vegList[i] = nullptr;
        }
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1> cd
grocery_list } ; if ($?) { .\grocery_list }
Fruit created: Apple
Fruit created: Banana
Vegetable created: Broccoli
Vegetable created: Lettuce

Jenna's Grocery List:
  [Fruit] Apple | Price: PHP 10 | Qty: 7 | Subtotal: PHP 70
  [Fruit] Banana | Price: PHP 10 | Qty: 8 | Subtotal: PHP 80
  [Vegetable] Broccoli | Price: PHP 60 | Qty: 12 | Subtotal: PHP 720
  [Vegetable] Lettuce | Price: PHP 50 | Qty: 10 | Subtotal: PHP 500

Total: PHP 1370
```

```
Removing Lettuce:
Vegetable deleted: Lettuce

Jenna's Grocery List (Updated):
  [Fruit] Apple | Price: PHP 10 | Qty: 7 | Subtotal: PHP 70
  [Fruit] Banana | Price: PHP 10 | Qty: 8 | Subtotal: PHP 80
  [Vegetable] Broccoli | Price: PHP 60 | Qty: 12 | Subtotal: PHP 720

Total (after removing Lettuce): PHP 870

Program terminating. Cleaning remaining memory:
Fruit deleted: Apple
Fruit deleted: Banana
Vegetable deleted: Broccoli
```

Code Analysis:

This program is like a digital shopping cart for a grocery list. It creates four items. Apple, Banana, Broccoli, and Lettuce, and shows them on the screen with their total cost. Next, it deletes Lettuce from the list to free up computer memory, which automatically updates the shopping list and lowers the total price. Finally, right before the program closes, it cleans up and deletes the remaining three items so that no computer memory is wasted.

Conclusion:

In conclusion, this project shows how to make a working shopping cart program using C++. I learned exactly how the computer creates items, tracks prices, and deletes food when it is no longer needed. I also learned that cleaning up memory with delete and using nullptr keeps the program safe from crashing. Even though managing computer memory can be tricky at first, this project gives me a great start toward mastering the C++ language.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization, checking if my idea is correct, explaining concepts, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **First I explain my concept idea then I let AI to clarify it.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.